



Chapter 2

Subtests (t.Run)

Turn every test case into an independent test.

- ✓ Cleaner output.
- ⚡ Faster debugging.

```
1 func TestAdd(t *testing.T) {
2     tests := []struct {
3         name string
4         a, b, want int
5     }{
6         {"positive", 1, 2, 3},
7         {"negative", -1, -2, -3},
8         {"zero", 0, 0, 0},
9     }
10    for _, tc := range tests {
11        t.Run(tc.name, func(t *testing.T) {
12            got := Add(tc.a, tc.b)
13            if got != tc.want {
14                t.Fatalf("got %d, want %d",
15                    got, tc.want)
16            }
17        })
18    }
19 }
```

```
=== RUN   TestAdd
    --- RUN   TestAdd/positive
        -- PASS: TestAdd/positive (0.00s)
    --- RUN   TestAdd/negative
        -- PASS: TestAdd/negative (0.00s)
    --- RUN   TestAdd/zero
        -- PASS: TestAdd/zero (0.00s)

PASS
ok      example.com/project    0.01s
```





1/7



The Problem

Table-Driven Tests reduce duplication, but every scenario still belongs to a **single test**.

```
func TestAdd(t *testing.T) {
    tests := []struct {
        name string
        a, b, want int
    }{
        {"positive", 1, 2, 3},
        {"negative", -1, -2, -3},
        {"zero", 0, 0, 0},
    }
    for _, tc := range tests {
        got := Add(tc.a, tc.b)
        if got != tc.want {
            t.Fatalf("got %d, want %d",
                got, tc.want)
        }
    }
}
```



If one fails, the whole test fails.
Harder to see which scenario actually failed.



Scenario:
positive



Scenario:
negative



Scenario:
zero



Single
Test



2/7



What Are Subtests?

`t.Run` gives every scenario its own identity, making tests easier to organize and understand.

```
func TestAdd(t *testing.T) {
    tests := []struct {
        name string
        a, b, want int
    }{
        { "positive", 1, 2, 3},
        { "negative", -1, -2, -3},
        { "zero", 0, 0, 0},
    }
    for _, tc := range tests {
        tc := tc // capture range variable
        t.Run(tc.name, func(t *testing.T) {
            got := Add(tc.a, tc.b)
            if got != tc.want {
                t.Fatalf("got %d, want %d",
                    got, tc.want)
            }
        })
    }
}
```

Parent Test

TestAdd

**t.Run**
"positive"a: 1, b: 2,
want: 3**t.Run**
"negative"a: -1, b: -2,
want: -3**t.Run**
"zero"a: 0, b: 0,
want: 0

Each scenario becomes a named, independent subtest.
Clear, focused, and easy to debug.



3/7



Meet t.Run

A simple function that creates a brand-new subtest with its own execution context.

```
func TestAdd(t *testing.T) {
    cases := []struct {
        name string
        a, b, want int
    }{
        {"positive", 1, 2, 3},
    }

    for _, tc := range cases {
        t.Run(tc.name, func(t *testing.T) {
            got := Add(tc.a, tc.b)
            if got != tc.want {
                t.Fatalf("got %d, want %d", got, tc.want)
            }
        })
    }
}
```



New subtest
Each call creates a separate test.



Isolated context
Runs with its own name and lifecycle.

t.Run(name, func(t *testing.T))

Registers and runs a subtest with the given name.



Small function.
Big improvement for test organization.





4/7

Table-Driven + Subtests

Wrap each table entry with `t.Run` to improve reporting without changing your test structure.

```
func TestAdd(t *testing.T) {
    tests := []struct {
        name string
        a, b, want int
    }{
        {"positive", 1, 2, 3},
        {"negative", -1, -2, -3},
        {"zero", 0, 0, 0},
    }
    for _, tc := range tests {
        tc := tc // capture range variable
        t.Run(tc.name, func(t *testing.T) {
            got := Add(tc.a, tc.b)
            if got != tc.want {
                t.Fatalf("got %d, want %d",
                    got, tc.want)
            }
        })
    }
}
```



Table Entry

Each row is a scenario



Subtest Wrapper

`t.Run` gives it a name and its own context



Better Output

Clear, focused results for every scenario

Without Subtests

✗ --- FAIL: TestAdd (0.00s)
add_test.go:23: got -2, want -3

✗ Hard to know which case failed.

With Subtests

✓ === RUN TestAdd/positive
--- PASS: TestAdd/positive (0.00s)

✗ === RUN TestAdd/negative
--- FAIL: TestAdd/negative (0.00s)
add_test.go:23: got -2, want -3

✓ === RUN TestAdd/zero
--- PASS: TestAdd/zero (0.00s)

★ Now it's obvious.
Each scenario stands on its own.



Same structure.
Better results.
Just one small change.



5/7

Better Test Output

Instead of **FAIL TestAdd**, you'll immediately know which scenario failed.

```
=== RUN    TestAdd/positive
--- PASS:  TestAdd/positive (0.00s)

✗ === RUN  TestAdd/negative
--- FAIL:  TestAdd/negative (0.00s)
    add_test.go:23: got -2, want -3

✓ === RUN  TestAdd/zero
--- PASS:  TestAdd/zero (0.00s)
```

✗ **Failed Scenario:** TestAdd/negative
Input: a = 1, b = -3
Expected: -3 | Got: -2



Clear failures.
Faster debugging.
Better confidence.



6/7

Why It Matters

Better reporting, easier debugging, cleaner hierarchy, and support for **selective & parallel** execution.



Better Reporting

Know **exactly** what passed, what failed, and why.



Easier Debugging

Instantly **identify** the failed scenario with input and actual output.



Cleaner Hierarchy

t.Run creates a **natural structure** that's easy to read and maintain.



Selective & Parallel Execution

Run **only what you need**.
Speed up feedback and CI.



Better tests.

Better code.

Better developer experience.



END

Read the Full Chapter

Learn Subtests (t.Run) in depth with examples, best practices, and common pitfalls.



Link in bio.

HANDBOOK

Table of Contents

- 01. Introduction
- 02. Subtests (t.Run)**
- 03. Test Structure
- 04. Best Practices
- 05. Common Pitfalls
- ...

02. Subtests (t.Run)

```
func TestAdd(t *testing.T) {
    tests := []struct {
        name string
        a, b, want int
    }{
        {"positive", 1, 2, 3},
        {"negative", -1, -2, -3},
    }
    for _, tc := range tests {
        tc := tc // capture range variable
        t.Run(tc.name, func(t *testing.T) {
            got := Add(tc.a, tc.b)
            if got != tc.want {
                t.Fatalf("got %d, want %d", got, tc.want)
            }
        })
    }
}
```

On this page

- Why Subtests?
- How t.Run Works
- Examples
- Best Practices
- Common Pitfalls
- ...



Github:

<https://github.com/golang-evangelist/handbook/.../02-subtests.md>



Level up your testing.
Better tests, better code.